Report

SE-2530: Baimagambet Gauhar

The purpose of this task was to compare the Binary Search Tree (BST) and a Hash Table.Both data structures support insertion, searching and deletion,but they organize data differently.
A Binary Search Tree stores smaller keys in the left subtree and larger keys in the right subtree.Its performance depends on the shape of the tree.If the tree is balanced,operations can take O(log n) time.In the worst case,an ordinary BST becomes unbalanced and operations take O(n) time.
A Hash Table uses a hash function to calculate where each key should be stored.In this task,separate chaining was used to handle collisions.When several keys are assigned to the same bucket,they are stored in a linked list.Hash Table operations usually take O(1) expected time, but in the worst case they can take O(n) time because of collisions.
The performance of both structures was tested using 100, 1,000, and 10,000 elements.The experiments measured insertion, searching and deletion times.Random input,sorted input for the BST and hash collisions were also considered.
The results demonstrate that input data can significantly affect performance.Hash Table usually performs well when keys are distributed evenly.However,many collisions can make operations slower.BST can perform efficiently when its shape is good,but inserting sorted keys into an ordinary BST can create a long chain and reduce performance.
In conclusion,a Hash Table is a good choice when fast key-based access is important and the order of elements is not required.A BST is useful when maintaining sorted data is important.The best data structure depends on the requirements of the application and the input data.